Sistemas Operacionais

Gerenciamento de Memória

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Avaliação Perfil do Ingressante – Católica SC

https://avaliacao.catolicasc.org.br/cpa/dashboard

Objetivos da Aula de Hoje

- **Compreender** o conceito de memória virtual como uma abstração criada pelo sistema operacional, entendendo como ela permite a criação de espaços de endereçamento isolados, independentes da memória física, garantindo segurança, multiprogramação e execução eficiente de processos.
- **Entender** o funcionamento da paginação e da tradução de endereços, analisando como a memória virtual é dividida em páginas, a memória física em quadros, e como a MMU utiliza a tabela de páginas para mapear endereços virtuais em endereços físicos.
- **Analisar** o impacto da TLB (Translation Lookaside Buffer) no desempenho do sistema, compreendendo seu papel como cache de traduções de endereços e avaliando como as taxas de acerto (hit) e falha (miss) influenciam o tempo de acesso à memória.
- **Comparar** os principais algoritmos de substituição de páginas (FIFO, LRU e Ótimo), examinando seu comportamento, vantagens, limitações e impacto no número de faltas de página e no desempenho geral do sistema.
- **Aplicar** os conceitos de memória virtual por meio de exercícios práticos, incluindo simulação de tradução de endereços e análise de algoritmos de substituição, com foco na interpretação de desempenho e tomada de decisão em cenários reais, tempo de espera e resposta) e análise de situações reais envolvendo sincronização e deadlock.

Memória Principal (RAM)

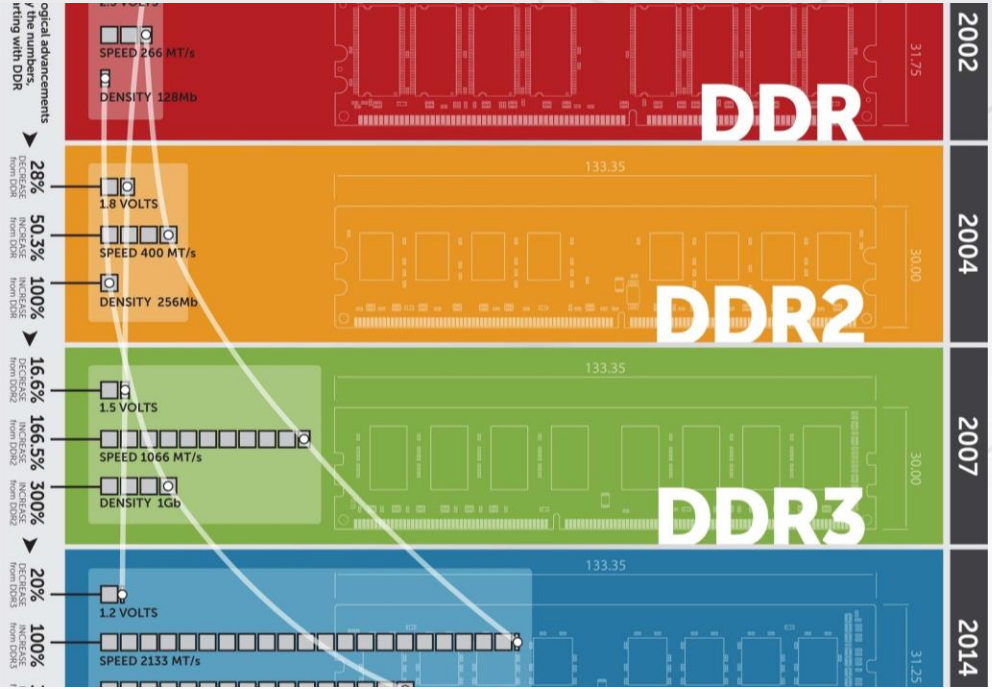
A memória principal, ou RAM (Random Access Memory), é o componente responsável por armazenar dados e instruções que estão sendo utilizados ativamente pela CPU.

A CPU acessa a RAM diretamente através do barramento de memória

- Cada instrução executada depende de acesso à memória

Processo simplificado

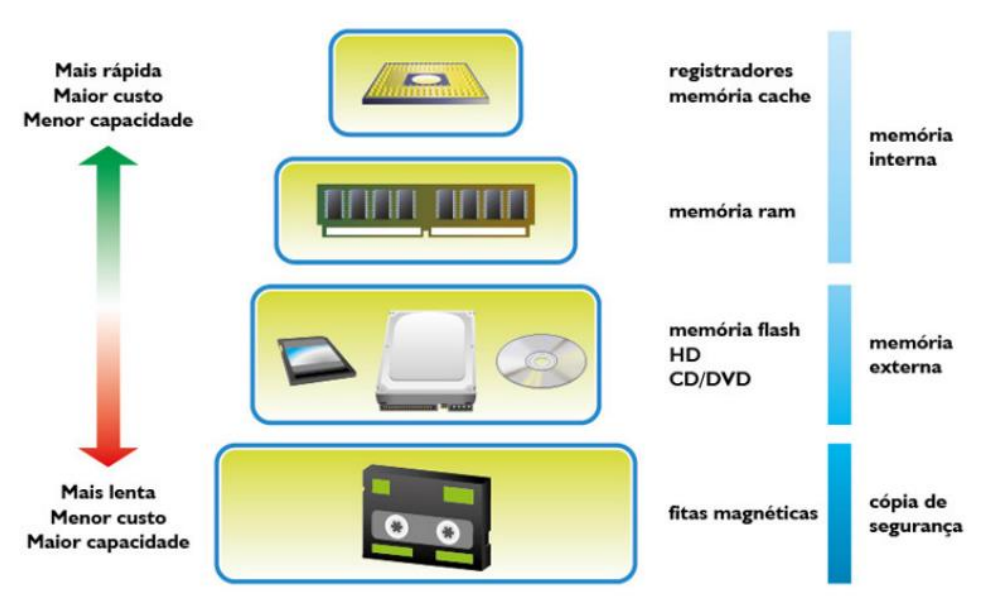
1. CPU busca instrução na RAM
2. Executa
3. Lê/escreve dados na RAM



Memória volátil e de alta velocidade

1. Volátil: perde os dados quando o computador é desligado
2. Alta velocidade: muito mais rápida que disco (HD/SSD)

- RAM → rápida, pequena, temporária
- Disco → lento, grande, permanente



Recurso limitado

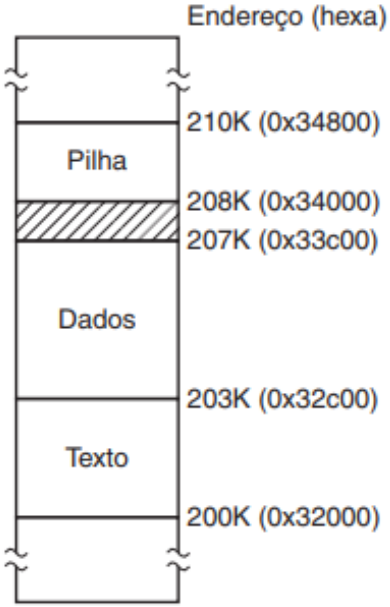
1. RAM é finita
2. Muitos processos competem por ela

Papel do sistema operacional

- Alocar memória para processos
- Liberar quando processos terminam
- Evitar conflitos
- Decidir o que fica na RAM

Sem sistema operacional:

- travamentos
- sobrescrita de memória
- falhas de segurança



(a)

	Virtual	Físico	Tamanho
Pilha	0x8	0xd0	0x2
Dados	0	0xc8	0x7
Texto	0	0xc8	0

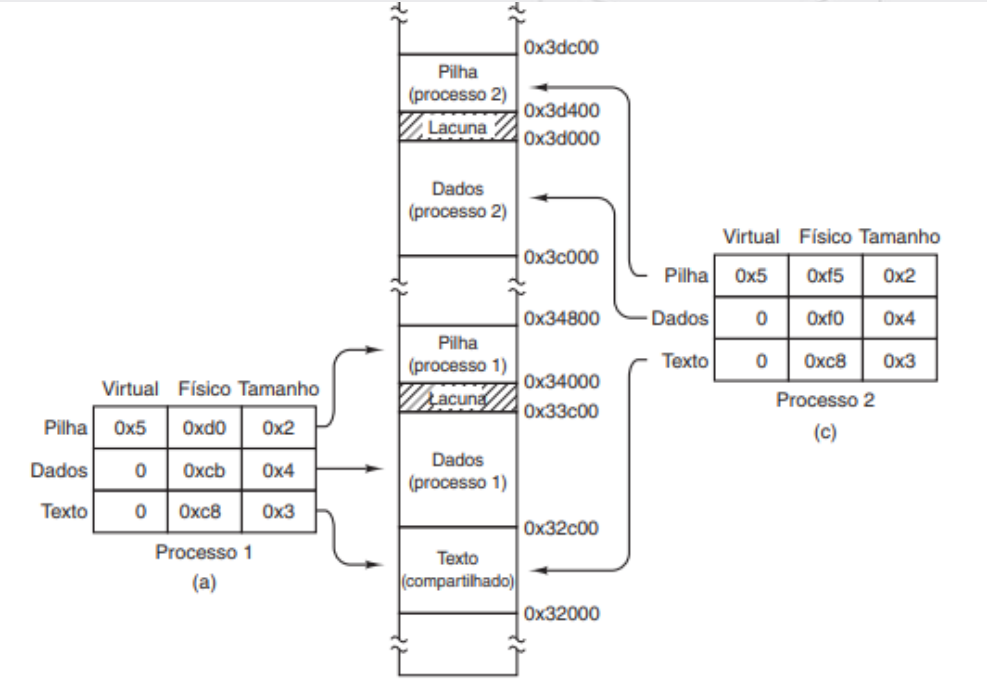
(b)

	Virtual	Físico	Tamanho
Pilha	0x5	0xd0	0x2
Dados	0	0xcb	0x4
Texto	0	0xc8	0x3

(c)

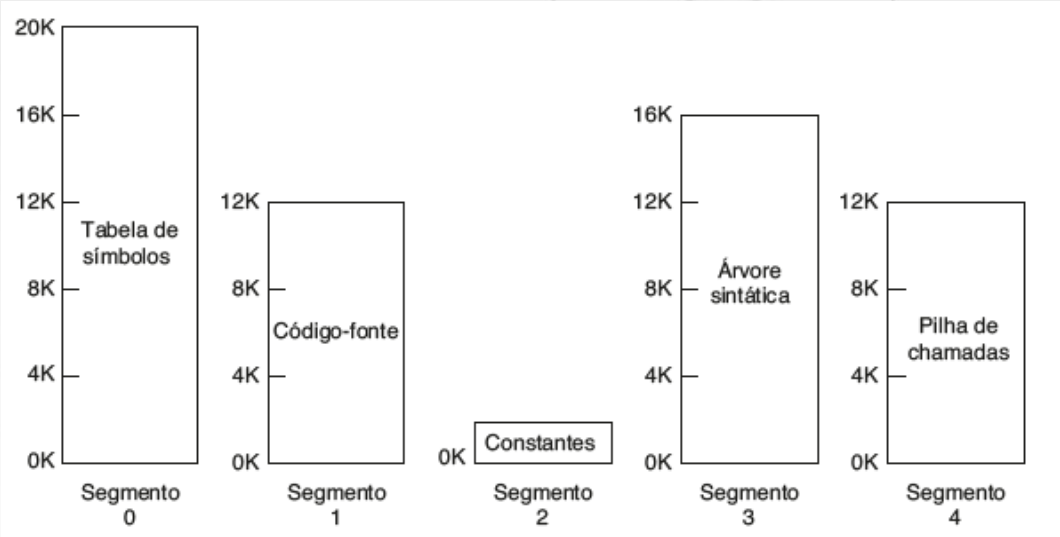
Memória Virtual

1. Abstração criada pelo sistema operacional
2. Cada processo possui espaço de memória próprio
3. Endereços virtuais independentes da RAM
4. Permite executar programas maiores que a memória física



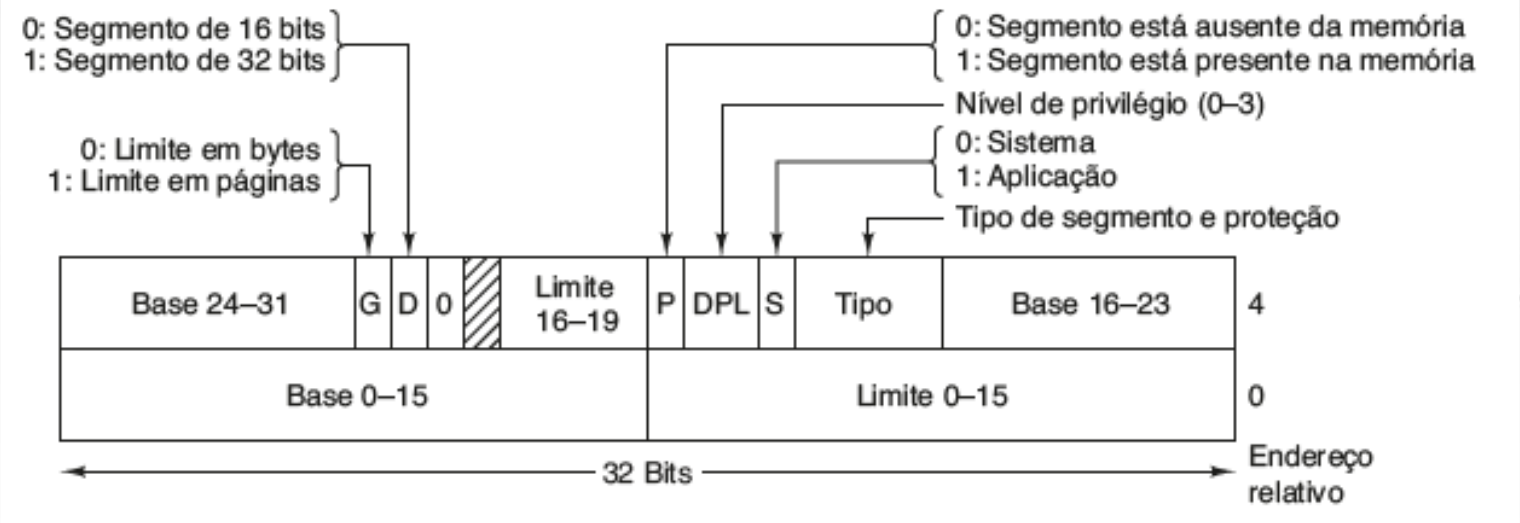
Espaço de Endereçamento

1. Conjunto de endereços que o processo enxerga
2. Endereços virtuais ≠ endereços físicos
3. Cada processo é isolado dos outros
4. Base para segurança e estabilidade



Espaço de Endereçamento

1. Cada processo trabalha com um conjunto próprio de endereços
2. Esses endereços vão de: 0 até um valor máximo (ex: 2^{32} ou 2^{64})
3. Esse espaço é chamado de espaço de endereçamento virtual



Paginação

A paginação é uma técnica de gerenciamento de memória em que o sistema operacional divide a memória em blocos fixos para facilitar a alocação e o controle.

1. Memória virtual dividida em páginas
2. Memória física dividida em quadros (frames)
3. Tamanho fixo (ex: 4 KB)
4. Permite alocação não contígua

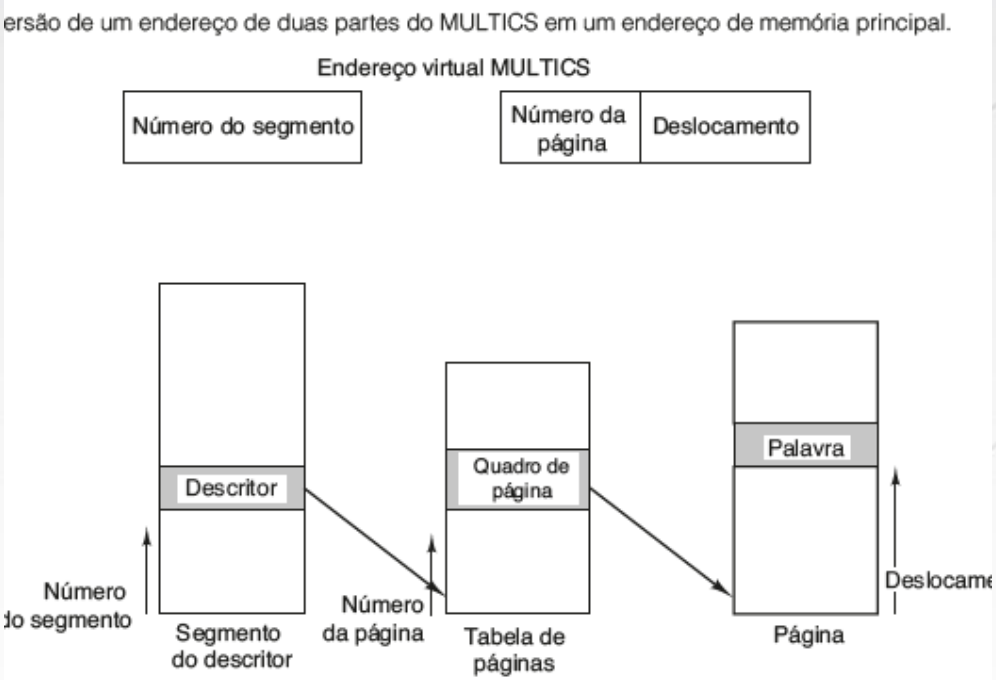
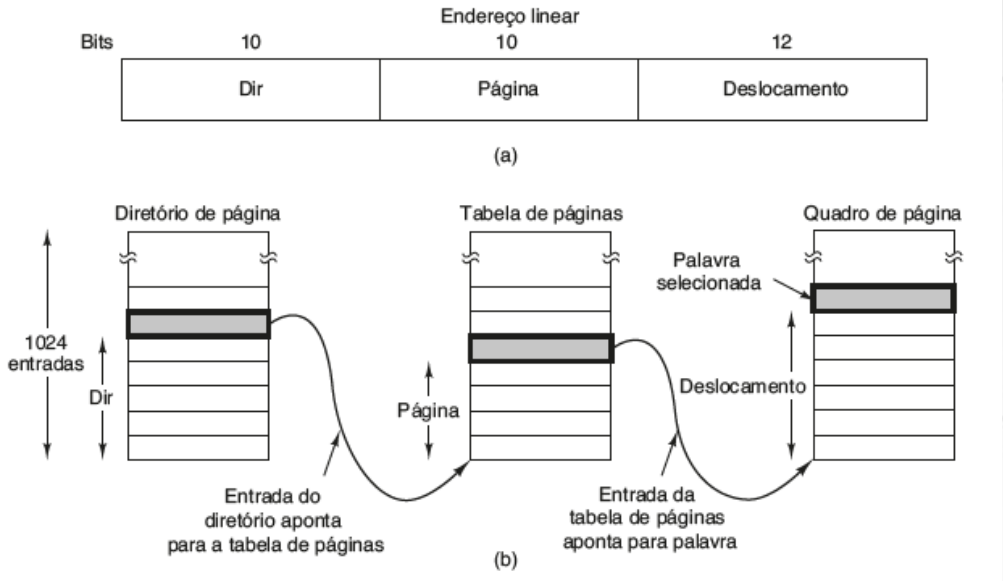


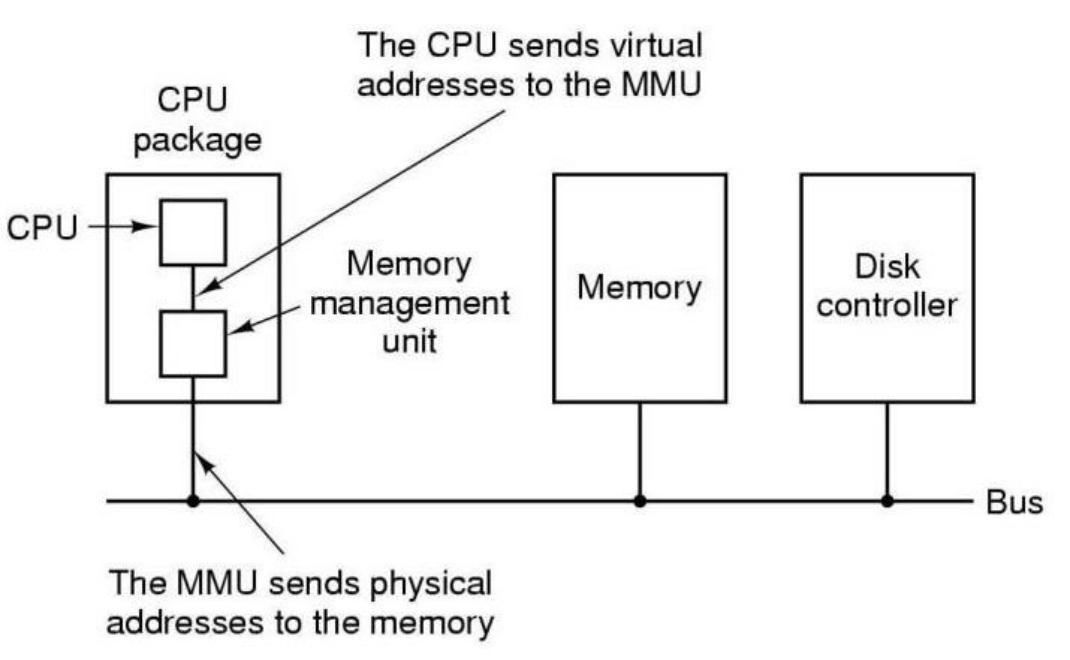
Tabela de Páginas

1. Mapeia páginas → quadros
2. Mantida pelo sistema operacional
3. Contém:
   - Bit de presença
   - Bit de modificação
   - Permissões de acesso



Tradução de Endereços

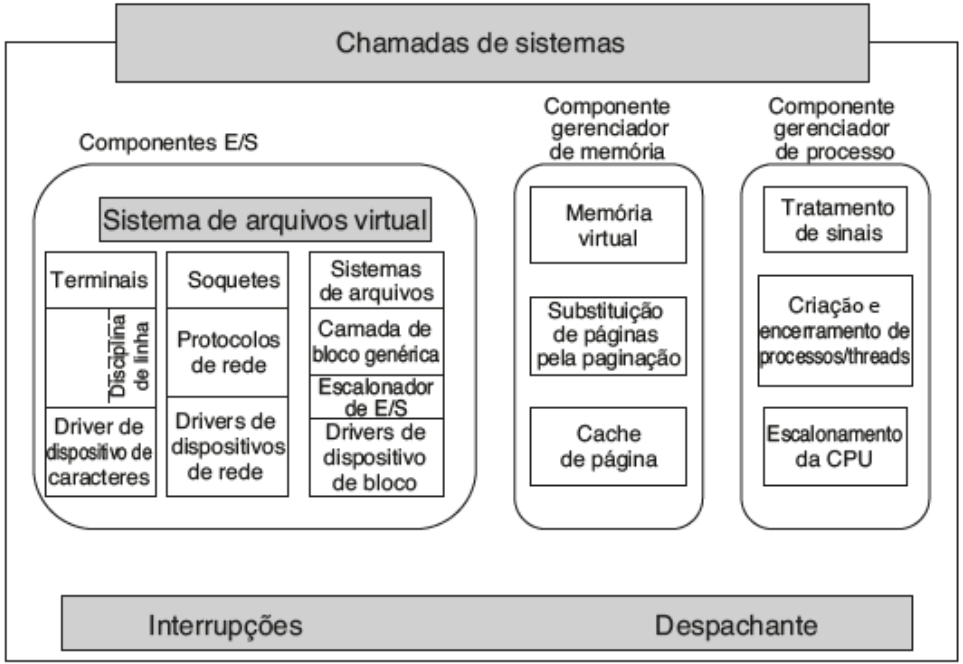
1. A tradução de endereços é o processo pelo qual o sistema transforma um endereço virtual (usado pelo processo) em um endereço físico (na RAM). (Automático, feito pelo SO com o **Memory Management Unit - MMU**)



CPU gera endereço virtual

1. Toda instrução executada pela CPU usa memória
2. A CPU sempre gera endereços virtuais, não físicos

- O programa não sabe onde está na RAM real



TLB (Translation Lookaside Buffer)

A TLB guarda mapeamentos já utilizados recentemente

Exemplo:

1. Página 2 → Quadro 7
2. Página 5 → Quadro 3

Esses mapeamentos ficam “prontos para uso”

Importante: Programas tendem a acessar as mesmas regiões de memória repetidamente

Válida	Página virtual	Modificada	Proteção	Quadro de página
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

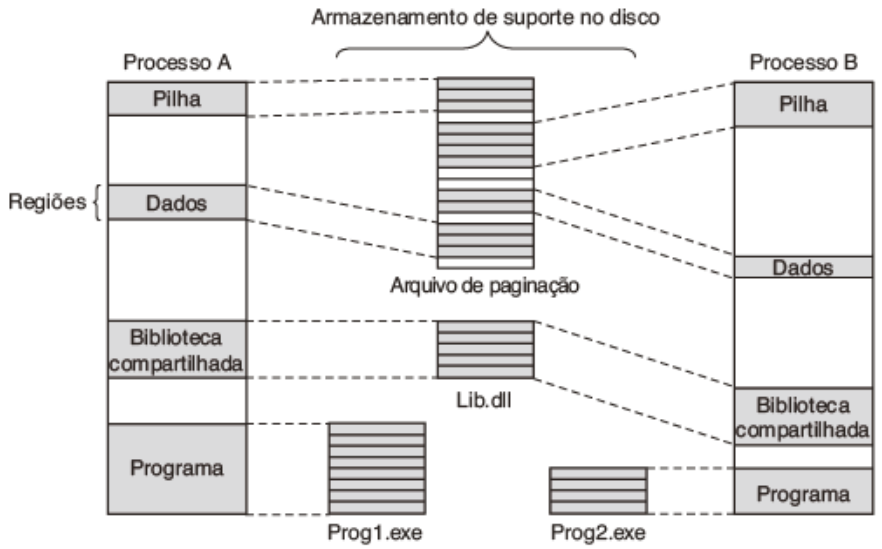
TLB (Translation Lookaside Buffer)

A TLB é muito menor que a RAM

- Usa hardware especializado (memória associativa)
- Permite busca extremamente rápida

- Acessar RAM → relativamente rápido
- Acessar tabela de páginas → mais lento (pode exigir múltiplos acessos)
- Acessar TLB → muito mais rápido

Regiões mapeadas com suas páginas duplicadas no disco. O arquivo *lib.dll* é mapeado em dois espaços de endereçamento ao mesmo tempo.



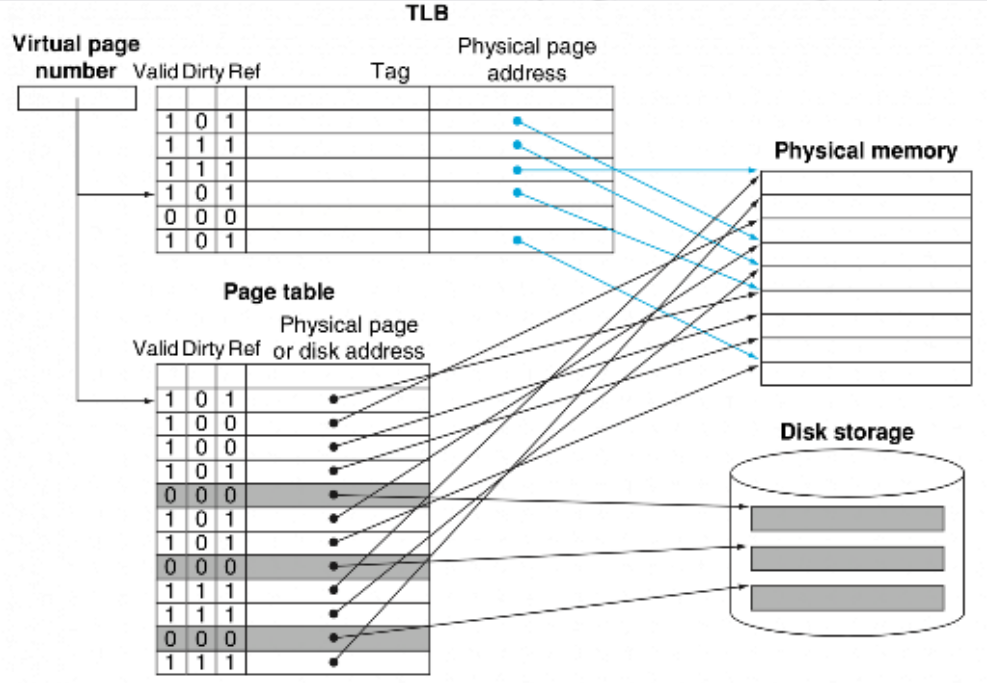
Por que usar a TBL?

Sem TLB:

- Cada acesso à memória exigiria:
  - acessar tabela de páginas
  - acessar o dado

Com TLB:

- Se a tradução estiver lá (TLB hit):
  - vai direto ao endereço físico
- Se não (TLB miss):
  - consulta tabela de páginas



TLB (Translation Lookaside Buffer) - Importante

A TLB faz parte da MMU (Memory Management Unit). Funciona automaticamente, sem intervenção do software na maioria dos casos.

Como funciona:

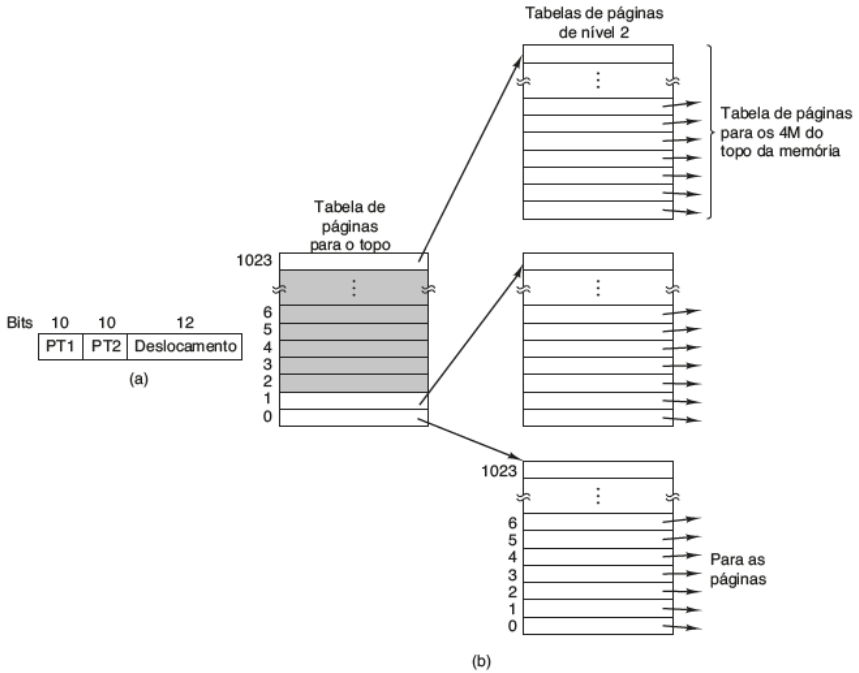
- CPU gera endereço virtual
- MMU consulta TLB
- Se encontrar → tradução imediata
- Se não → busca na tabela de páginas

➢ TLB Hit → tradução encontrada → rápido

➢ TLB Miss → precisa consultar tabela → mais lento

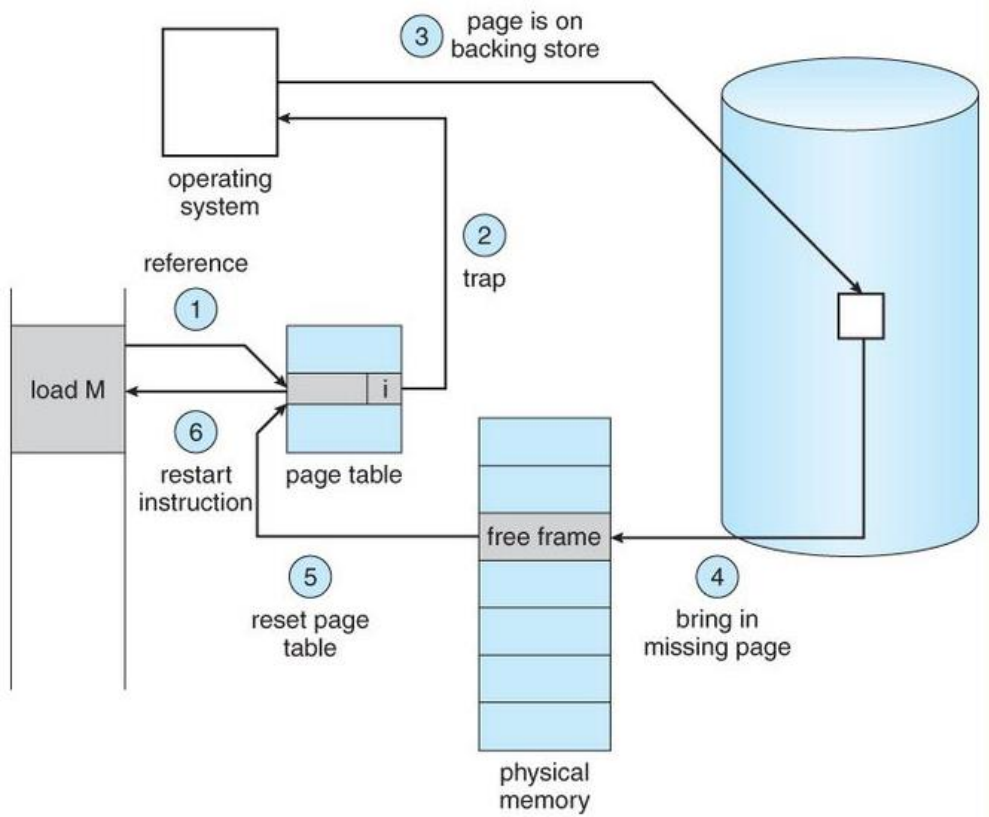
Page Fault & Tabelas de páginas multinível

- Ocorre quando uma página referenciada não está na memória RAM
- A MMU detecta a ausência e gera uma interrupção (**trap**) para o sistema operacional
- O sistema operacional localiza a página no disco (memória secundária)
- A página é carregada para um quadro livre na memória física
- A tabela de páginas é atualizada com o novo mapeamento
- A instrução que causou a falha é reexecutada
- A execução do processo continua normalmente



Fluxo de Page Fault

1. Processo acessa página
2. Página não está na memória (**trap**)
3. SO é acionado
4. Página é carregada do disco
5. Tabela é atualizada
6. Instrução é reexecutada



Substituição de Páginas

A substituição de páginas é o mecanismo utilizado quando a memória RAM está cheia e o sistema precisa trazer uma nova página para a memória, removendo uma página existente.

Exemplo: RAM cheia → precisa liberar espaço

- A memória física (RAM) tem capacidade limitada
- Quando ocorre um page fault, pode não haver quadros livres
- Nesse caso, o sistema precisa liberar espaço

➢ O sistema operacional deve decidir qual página será retirada da memória

➢ Essa decisão é feita por um algoritmo de substituição

Algoritmos de substituição

- Página mais antiga (FIFO)
- Página menos usada recentemente (LRU)
- Página usada mais tarde no futuro (Ótimo – teórico)

Algoritmo FIFO (First In, First Out)

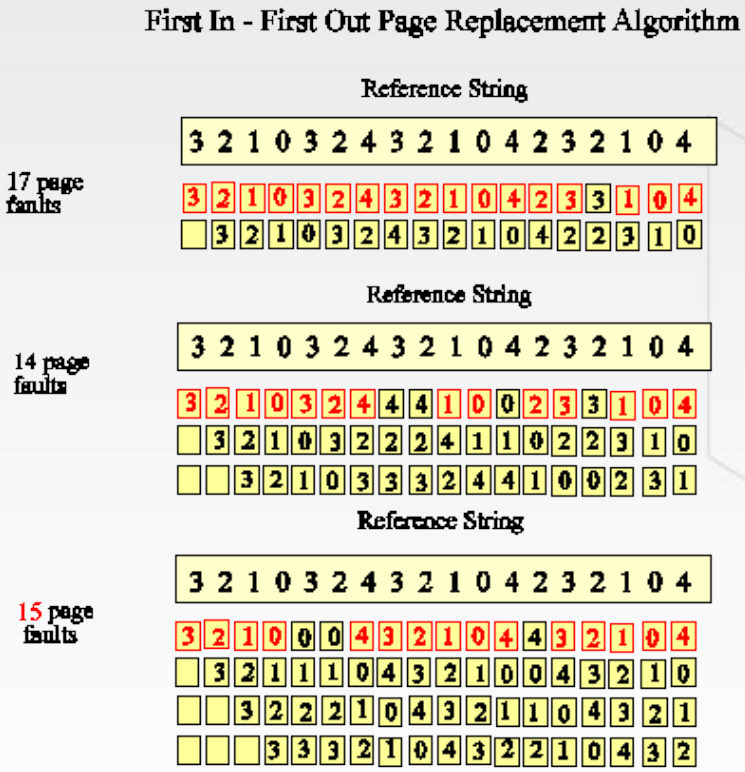
O “primeiro a entrar, primeiro a sair”

- A página que está há mais tempo na memória é a primeira a ser removida
- Utiliza uma estrutura lógica de fila (queue)
- Cada nova página entra no final da fila
- A página mais antiga (início da fila) é selecionada para substituição

➢ Pode ocorrer o fenômeno de Anomalia de Belady:

  - Aumentar a quantidade de memória pode aumentar o número de page fault

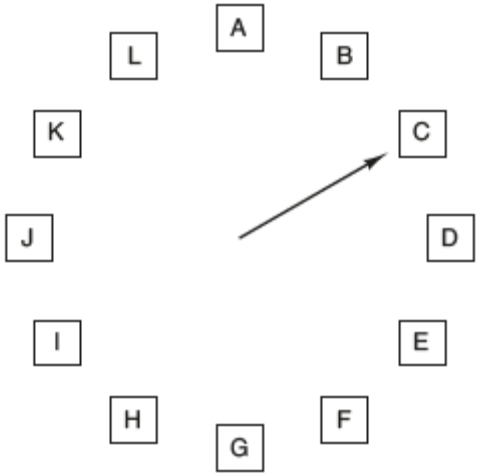
Algoritmo FIFO (First In, First Out) - Belady



Algoritmo LRU (Least Recently Used)

O “menos recentemente utilizado”

- A página que está há mais tempo sem ser acessada é escolhida para remoção
- Considera o histórico de uso das páginas
- Aproxima o comportamento ideal (algoritmo ótimo)



Quando ocorre uma falta de página, a página indicada pelo ponteiro é inspecionada. A ação executada depende do bit R:

R = 0: Remover a página
R = 1: Zerar R e avançar o ponteiro

Algoritmo Ótimo (Optimal)

- Baseado no princípio de remover a página que será utilizada mais tarde no futuro
- Para cada decisão, escolhe a página cujo próximo uso está mais distante no tempo
- Serve como referência teórica para comparação com outros algoritmos

Ao ocorrer necessidade de substituição:

- Observa todas as páginas na memória
- Verifica quando cada uma será usada novamente
- Remove aquela com uso mais distante (ou que não será mais usada)

Algoritmo intuitivo: Se você soubesse o futuro, removeria aquilo que demorará mais para ser necessário

Conjunto de Trabalho (Working Set)

Representa o conjunto de páginas que um processo utiliza ativamente em um determinado período de tempo

- Baseado no princípio de localidade de referência:
  - Programas tendem a acessar **um mesmo conjunto de dados repetidamente em curtos intervalos de tempo**
- Define quantas páginas um processo realmente precisa na memória para executar eficientemente

➢ O sistema observa os acessos recentes do processo

➢ Considera uma janela de tempo (Δ)

➢ Inclui no conjunto todas as páginas acessadas nesse intervalo

Conexão com Sistemas Atuais

- Sistemas como Linux utilizam paginação multinível, permitindo gerenciar grandes espaços de endereçamento com eficiência e menor uso de memória para tabelas de páginas
- Sistemas Windows também implementam memória virtual paginada, com mecanismos avançados de proteção, compartilhamento de memória e gerenciamento dinâmico
- CPUs modernas possuem suporte direto à memória virtual, incluindo MMU e TLB em hardware, tornando a tradução de endereços extremamente rápida
- O kernel do sistema operacional aplica otimizações complexas, como:
  - políticas avançadas de substituição de páginas
  - gerenciamento de Conjunto de Trabalho (working set)
  - uso de páginas grandes (huge pages)
  - técnicas para reduzir page faults e melhorar desempenho
- Esses mecanismos permitem que sistemas atuais executem múltiplos processos simultaneamente, com segurança, isolamento e alto desempenho

Hora de perguntar, dúvidas

Próxima aula

- Sistemas de Arquivos: organização de arquivos e diretórios, estrutura hierárquica (árvore), nomes de caminhos e operações básicas de manipulação de arquivos
- Métodos de Acesso: acesso sequencial, direto e indexado, análise de desempenho e aplicações em sistemas reais
- Alocação de Blocos: alocação contígua, encadeada e indexada, fragmentação e impacto no desempenho do sistema
- Gerenciamento de Espaço Livre: bitmap, listas encadeadas e técnicas modernas de controle de espaço em disco
- Atividade prática: construção de uma estrutura de diretórios em árvore e análise comparativa de como sistemas reais (Linux e Windows) organizam arquivos